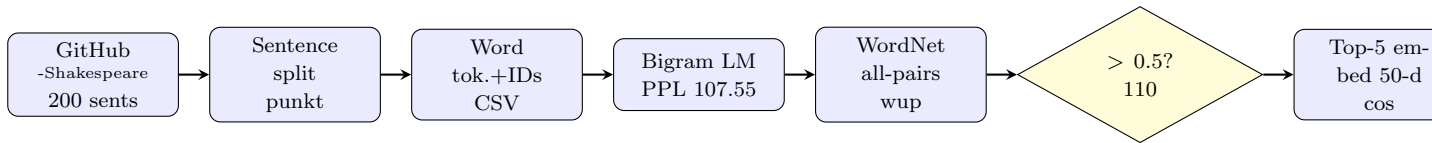


Assignment-1: Shakespeare → LM → WordNet → Embeddings

Group: _____ Karpathy tiny-Shakespeare (GitHub) | 20k chars |
200 sents | 3272 toks | V=339



Flow: dataset → sentences → tokens/IDs → LM → WordNet → threshold → embeddings.

1. Dataset: download + sentence split

```
urllib.request.urlretrieve(URL, "data/tinyshakespeare.txt")
sents = sent_tokenize(open(...).read()[:20000])[:200] # NLTK punkt
```

2. Tokenize → vocab → IDs (→ token_ids.csv)

```
from nltk.tokenize import word_tokenize
toks = word_tokenize(sent.lower()) # keep tokens with [A-Za-z]
tok.build_vocab(tokenized) # count, drop freq<2
# word2id = {<pad>:0, <unk>:1, the:2, you:3, ...} (339 types)
ids = [tok.word2id.get(t, 1) for t in toks] # e.g. [hear,me,speak]->[43,1,93]
# CSV row: sentence_id, sentence, "toks...", "ids...", length
```

3. Bigram LM with add-1 (→ language_model.json)

```
for sent in tokenized: # OOV already mapped to <unk>
    for i, w in enumerate(sent):
        unigram[w] += 1
        if i > 0: bigram[(sent[i-1], w)] += 1
P(w | prev) = (C(prev, w) + 1) / (C(prev) + V) # V = 339
PPL = exp(-mean(log P)) # train PPL = 107.55
```

4. WordNet all-pairs similarity, keep > 0.5 (→ wordnet_similarity.csv)

```
from nltk.corpus import wordnet as wn
from itertools import combinations
# candidates: top-80 frequent, alpha, len>2, no stopwords
syn = lambda w: wn.synsets(w.lower())
def score(a, b): # first synset = frequent sense
    A, B = syn(a), syn(b)
    return A[0].wup_similarity(B[0]) if A and B else None # path gives ~0!
kept = [(a, b, round(s, 4)) for a, b in combinations(words, 2)
        if (s := score(a, b)) and s > 0.5] # 110 pairs, sort desc
```

5. Top-5 pairs → co-occurrence embeddings (→ top5_embeddings.csv)

```
targets = unique(words of top-5 pairs) # 10 words: done,made,man,sir,...
for sent in tokenized: # window = 2, symmetric
    f = [t for t in sent if t in targets]
    for i, w in enumerate(f):
        for j in range(max(0,i-2), min(len(f),i+3)):
            if i != j: mat[w, f[j]] += 1
emb = mat / ||mat|| # L2 rows -> project to 50-d
cos(a, b) = a.b / (||a|| * ||b||) # e.g. sir-like 0.837
```

6. Results (final iter5_final)

w1	w2	wup			
done	made	1.000	a	b	cos
man	sir	0.947	sir	like	0.837
city	rome	0.900	man	like	0.739
like	make	0.900	done	like	0.704
people	poor	0.889			

7. Five iterations (same 200 sents).

#	tag	change	PPL	> 0.5
1	baseline	whitespace, keep case, $k=0$, path, rand-16d	7.18	1
2	hygiene	lower+NLTK, min_freq 2, $k=1$, stop-filter	107.55	0
3	wordnet	path → wup (0→110 pairs)	107.55	110
4	cooccur	random→co-occurrence, window 2, 50-d	107.55	110
5	final	freeze + cosine heatmap + stats	107.55	110

Run (venv only): make venv && make install && make test && make run **Out:** token_ids.csv, vocab.json, language_model.json, wordnet_similarity.csv, top5_embeddings.csv/npy, stats.json.